Software Engineering

Dr. Fatma ElSayed

Dr. Ahmed Yousry

Computer Science Department

Course Information

Contents

- Introduction to Software Engineering
- Software Processes
- Requirements Engineering
- System Modelling Part1
- System Modelling Part2
- System Architecture
- Software Testing Strategies
- Software Testing Techniques
- Technical Metrics for Software

Course Information

Contents

- Introduction to Software Engineering
- Software Processes
- Requirements Engineering
- System Modelling Part1
- System Modelling Part2
- System Architecture
- Software Testing Strategies
- Software Testing Techniques
- Technical Metrics for Software

Activity Diagram

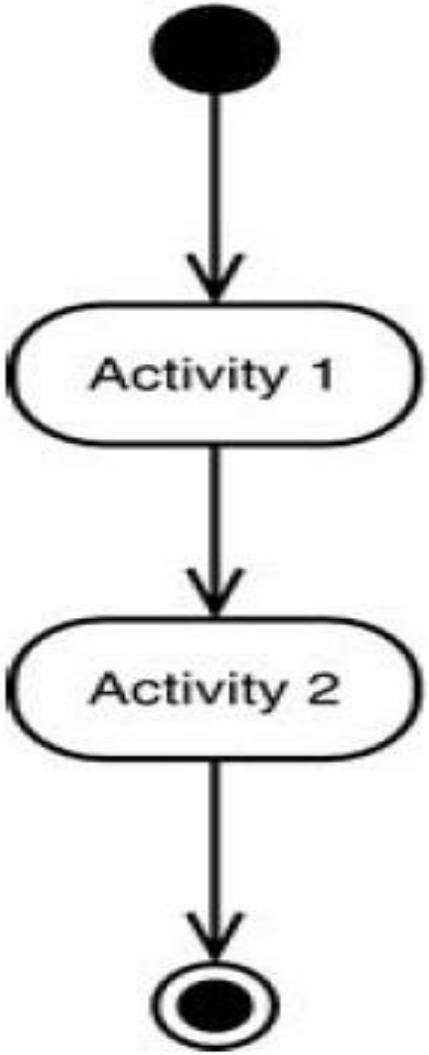
Activity Diagrams

- Activity diagram gives a simplified, step-by-step view of what the system does during an operation or process *(Process perspective)*.
- This diagram much like the flowcharts . It shows steps ***(called, activities)***, showing their sequences, and where activities can be carried out in parallel as well as decision points and branches.
- It can also be extended to show which ***person*** is responsible for which activity.
- An activity could be a task that a *person* **or** *a computer* might perform.

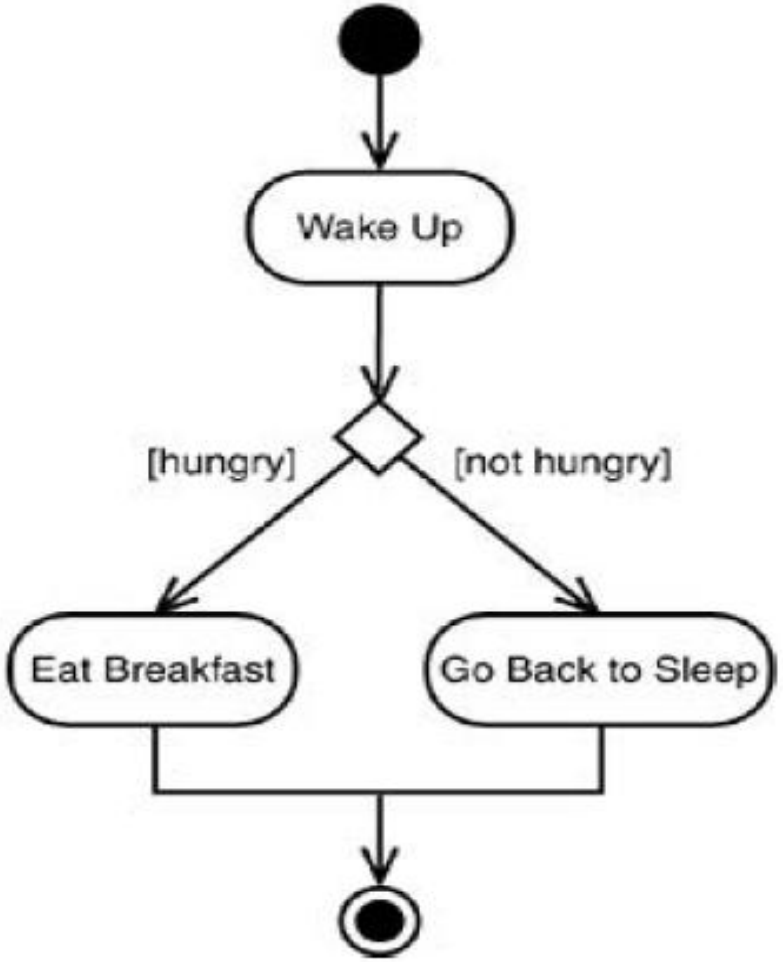
Activity Diagrams: Symbols and Elements

Element	Meaning	Symbol
Activities	Business processes and tasks	
Start node	Start of process	
End node	End of process	
Transition	move from one activity to another	
Synchronization bar	represents **fork** and **join**. Where **fork** represents the concurrent activities, and **join** means end of concurrent activities and is used to move to next sequence activity after finishing concurrent ones.	
Decision node	represent alternative ways out of an activity. It has 2 outgoing arcs	
Merge node	brings together alternative flow. It has one outgoing flow.	
Guard	represents the condition (Boolean test); Guards are associated with transitions. A transition takes place only if its guard evaluates to true at the time the flow of control reaches that transition.	[*condition*]
Swimlane	Swimlanes group **activities** associated with different **roles**. Each swimlane shows who is responsible for which set of related activities.	

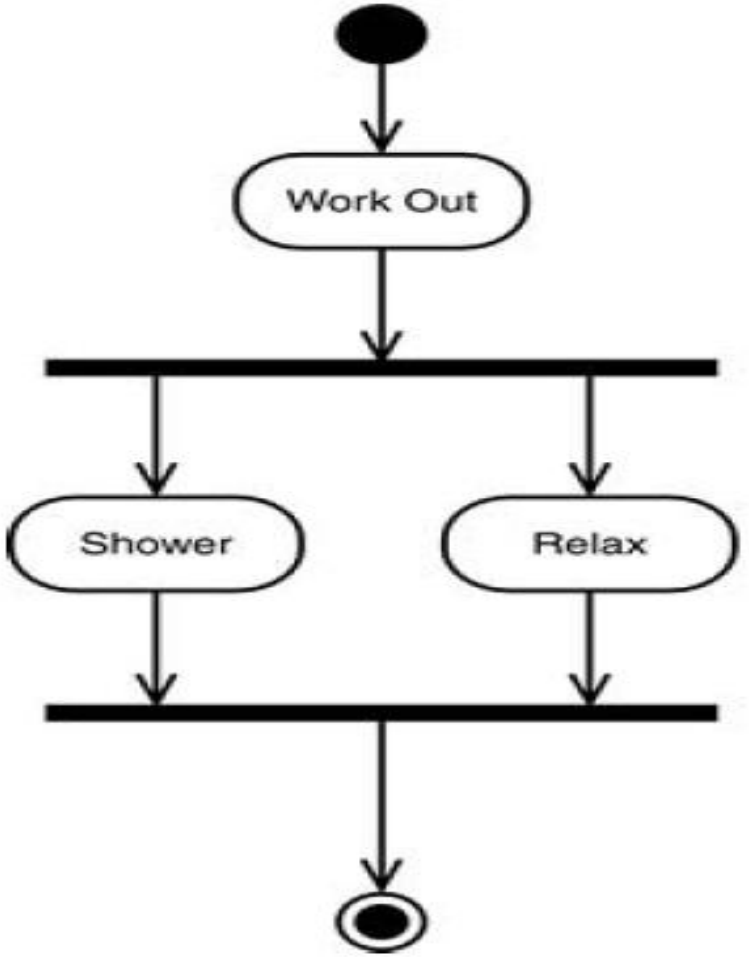
Activity Diagrams: Symbols and Elements



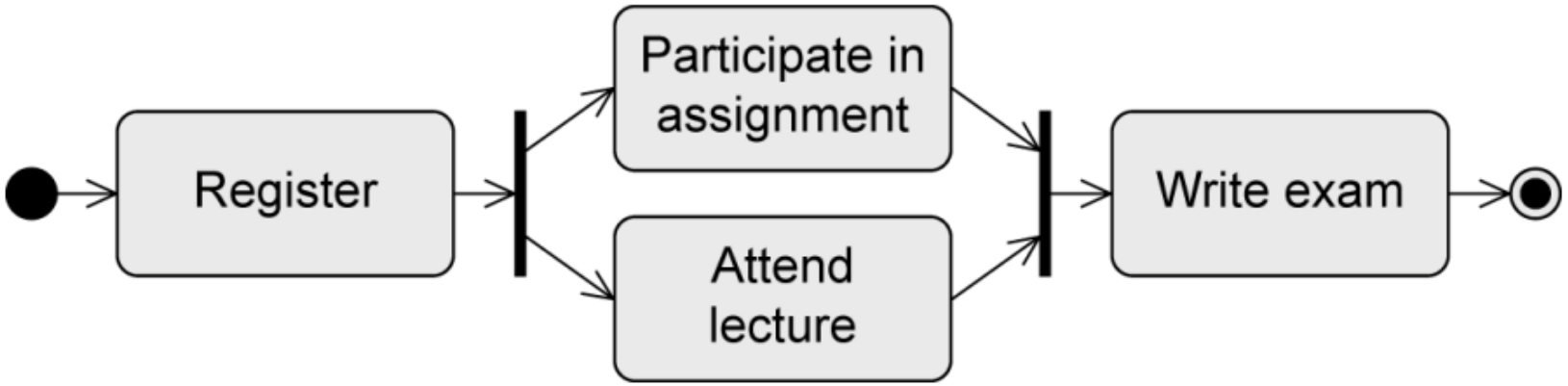
Decisions



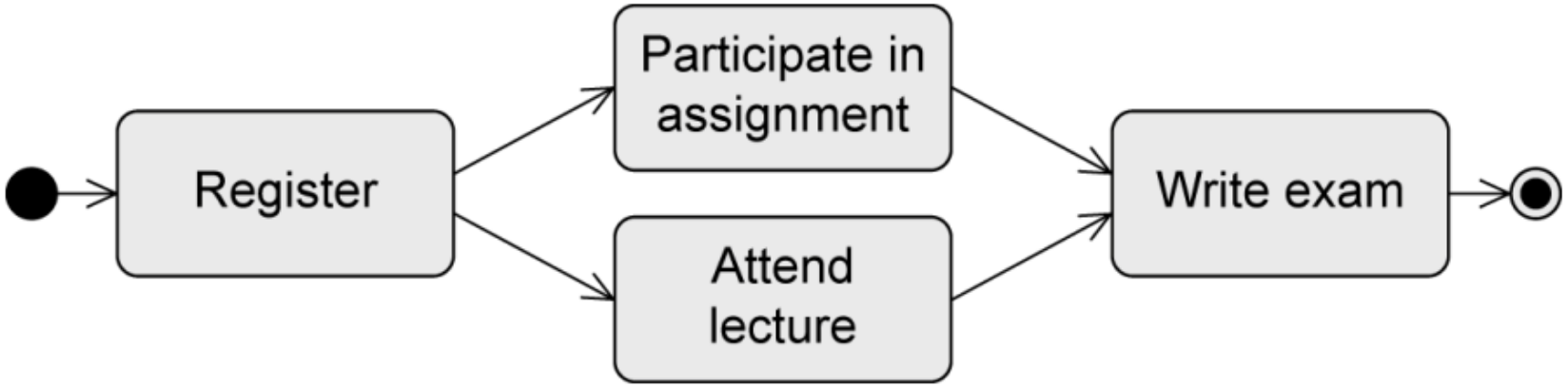
Concurrent Paths



Activity Diagrams: Symbols and Elements



.. equivalent to



Activity Diagrams: Example (1)

1) Suppose, after a hard day's work, you decide to make yourself a hot drink. What do you do to achieve that goal? What are the actions and how do you organize them?

We start with a **process** called make hot drink. At the center of this task is boiling water in a kettle.

Usually, there are two steps.

1. Fill the kettle with water.
2. Boil the water.

To begin with, we will consider instant coffee (as a particular scenario). Finally, you need a cup to drink from.

Activity Diagrams: Example (1)

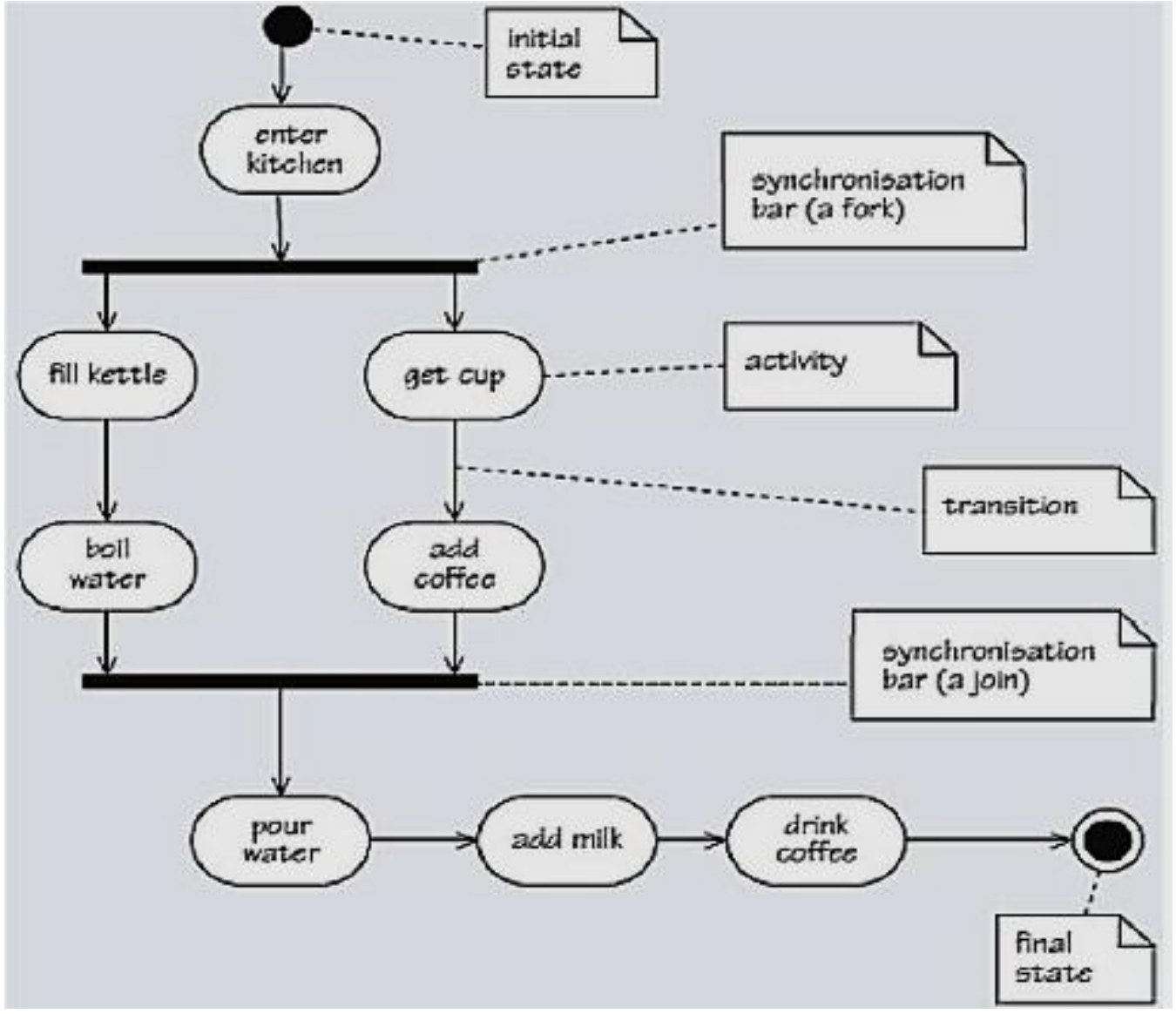


Figure 1

Activity Diagrams: Example (1)

The basic elements of an activity diagram are activities and transitions. The completion of an activity triggers an outgoing transition. Similarly, once the incoming transition has been triggered an activity may commence.

- There are two predefined activities, start and stop, which is represented as filled circles, a diagram may include more than one stop node if it makes it clearer.
- The Figure shows two synchronization bars, denoted by thick horizontal lines, the upper one denotes a fork and the lower one a join. The upper bar allows for separate activities to be carried out in parallel, so between the two synchronization bars the two separate strands can be performed concurrently.
- In UML, a note (with annotation) is shown as a rectangle with the top right-hand corner folded down.
- A dashed line is used to attach the note to the model element to which it refers.

Activity Diagrams: Example (1)

Redraw the activity diagram of preparing a cup coffee so that

- It allows to check if cup is clean **continue** the process, if not **wash** the cup.

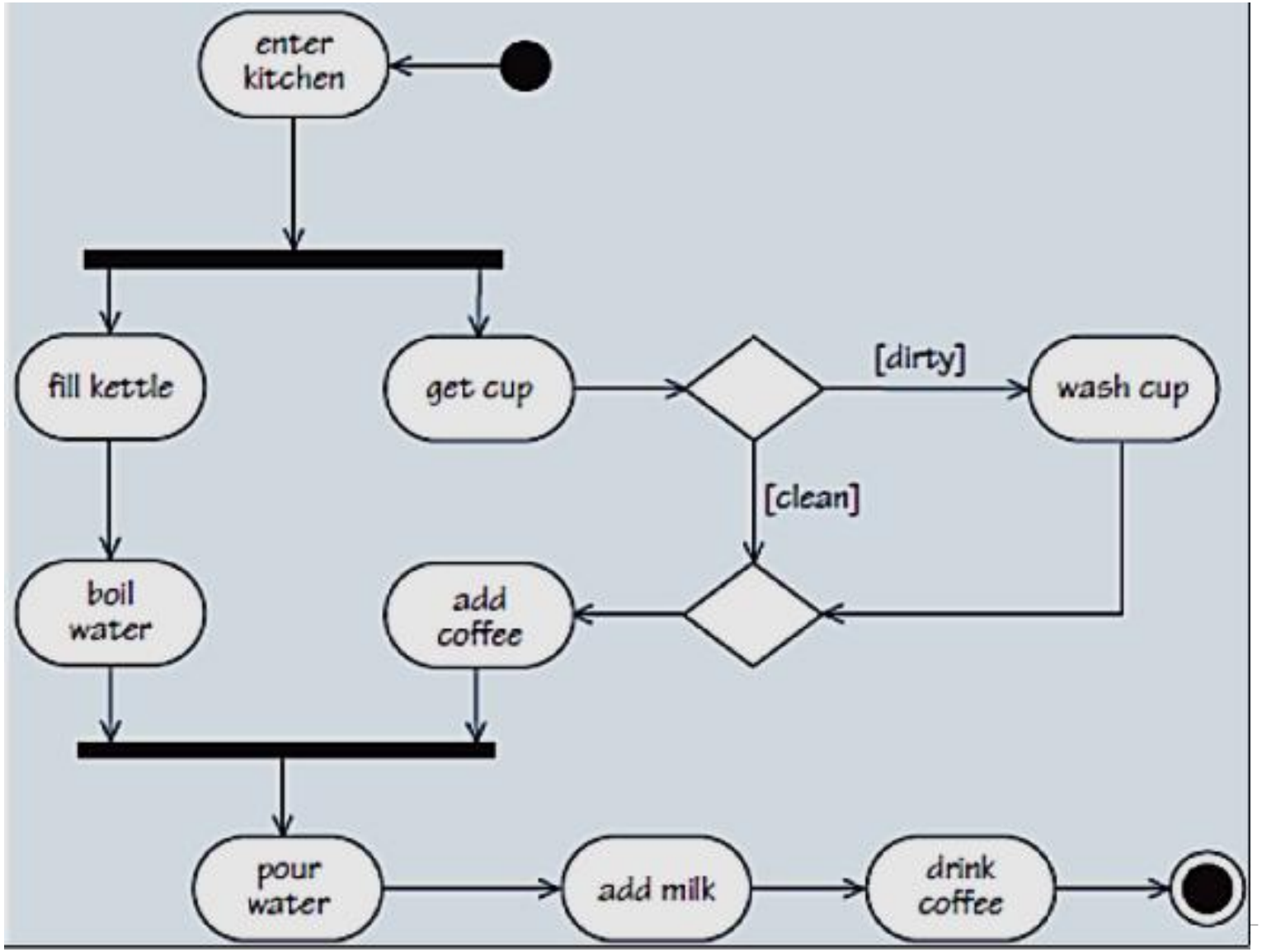


Figure 2

Activity Diagrams: Example (1)

- Each of the transitions leaving the first decision diamond has a guard to determine which path should be taken under a given condition.
- One outgoing transition should always be taken – this means that exactly one of the relevant guards must always evaluate to true.
- A diamond shape is also used as a merge node, which brings together alternative mutually exclusive flows, as shown in figure 2.
- A merge node will be reached only by one of the alternative flows and has a single outgoing flow.
- This is enforced by ‘decision’ and ‘merge’ nodes for alternative outgoing and incoming transitions, and by synchronization bars for concurrent outgoing (fork) and incoming (join) transitions

Activity Diagrams: Example (2)

2) Consider the lending process from a library :

A typical lending library keeps a stock of books for the use of its members. **Each member** can look on the library shelves to select a copy to borrow. Then take out a number of books, up to a certain limit. After a given period of time, the library expects members to return the books that they have on loan.

When borrowing books, a member is expected to wait in queue, then to hand their chosen books to the **librarian**, who records each new loan before issuing the books to the member. After that the librarian will prepare for next member in the queue.

Activity Diagrams: Example (2)

An activity diagram of **borrowing a copy** of a book from the library.

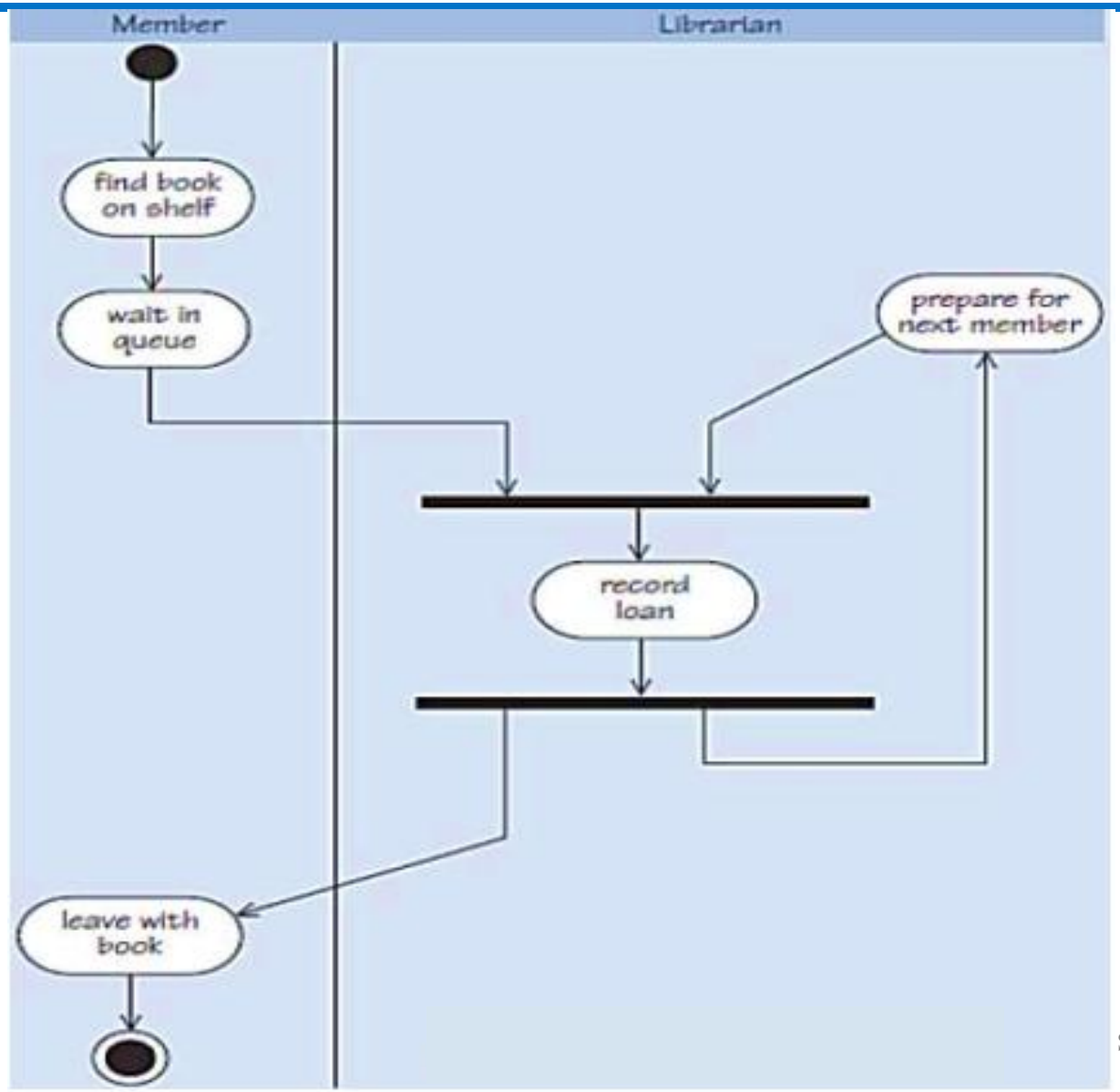


Figure 3

Activity Diagrams: Example (2)

Swim-lanes group activities associated with different roles.

- The swim-lanes show the role that is responsible for each activity.
- When you are modelling a workflow that involves more than one role, it is possible to identify which role is responsible for a particular activity.
- An activity diagram can help identify the stages at which each role requires some interaction with the process.

In figure 3: The left-hand swim-lane contains the activities that a library member performs when borrowing a book, while the right-hand swim-lane shows the library's self-service system workflow for the same process ('issue copy of book').

Activity Diagrams: Swim lanes

- The activity diagram adds the dimension of visualizing roles.
- To do that, you separate the diagram into parallel segments.
- Each segment shows the name of a role at the top and presents the activities of each role.
- Transitions can take place from one segment to another.

Activity Diagrams: Example (3)

An activity diagram of **returning a copy** of a book to the library.

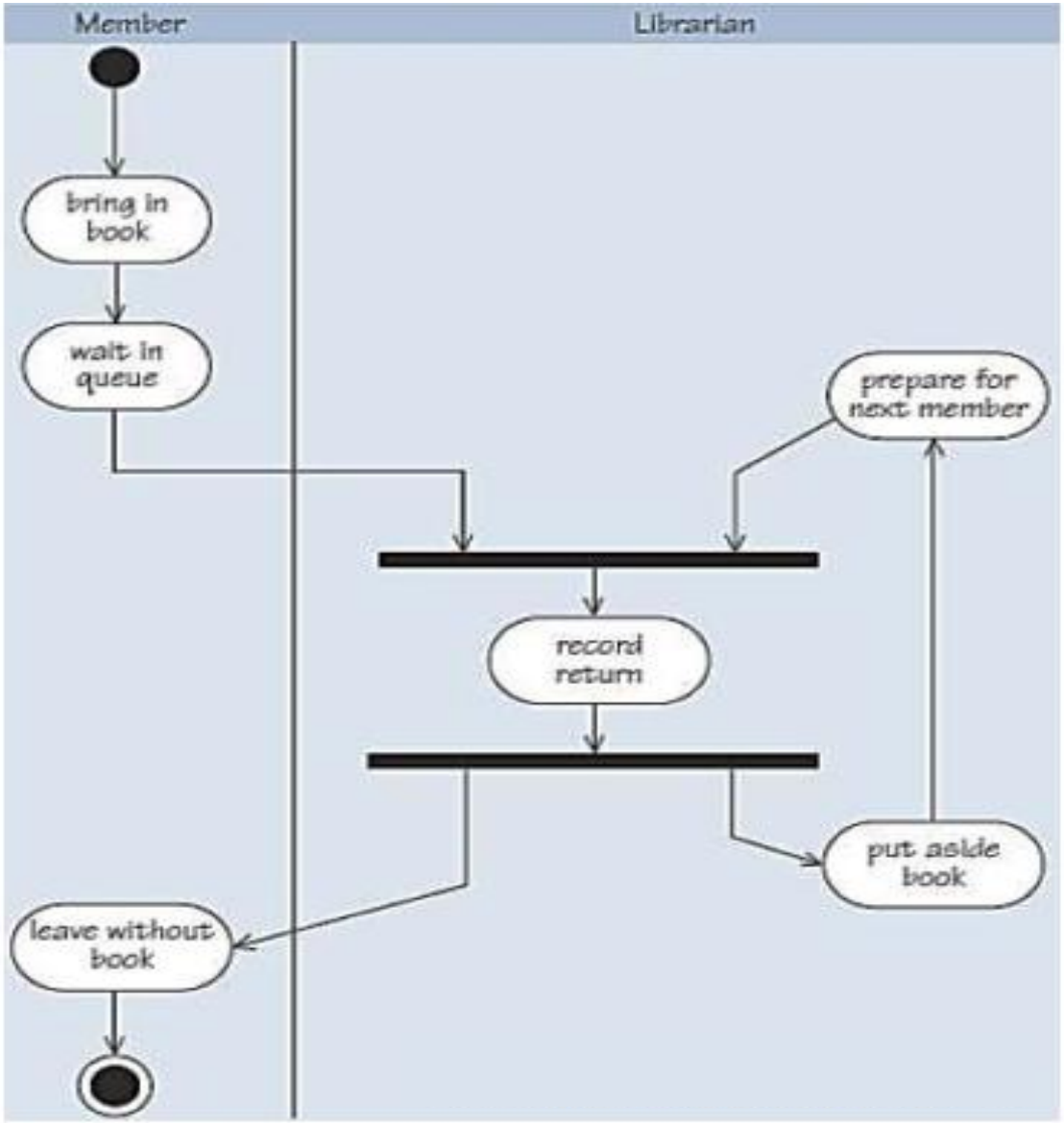
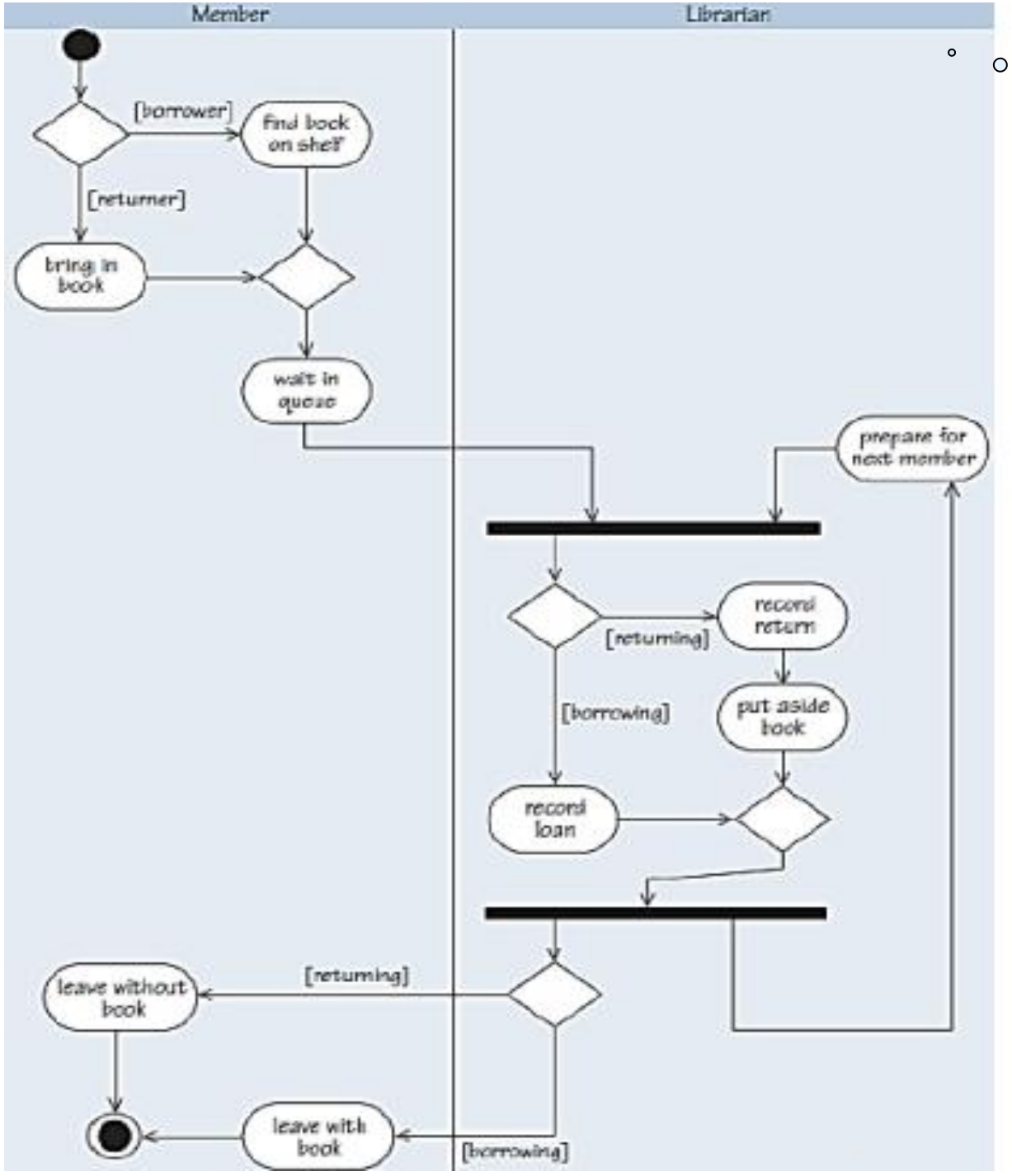


Figure 4

Example (4)

An activity diagram of **borrowing and returning** a copy of a book.



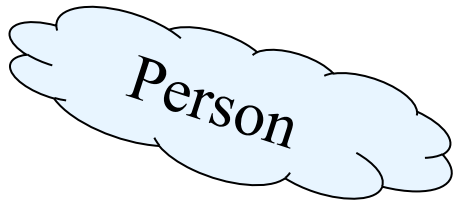


Figure 5

Example (5)

Activity Diagram for process of assessing university students via exams.

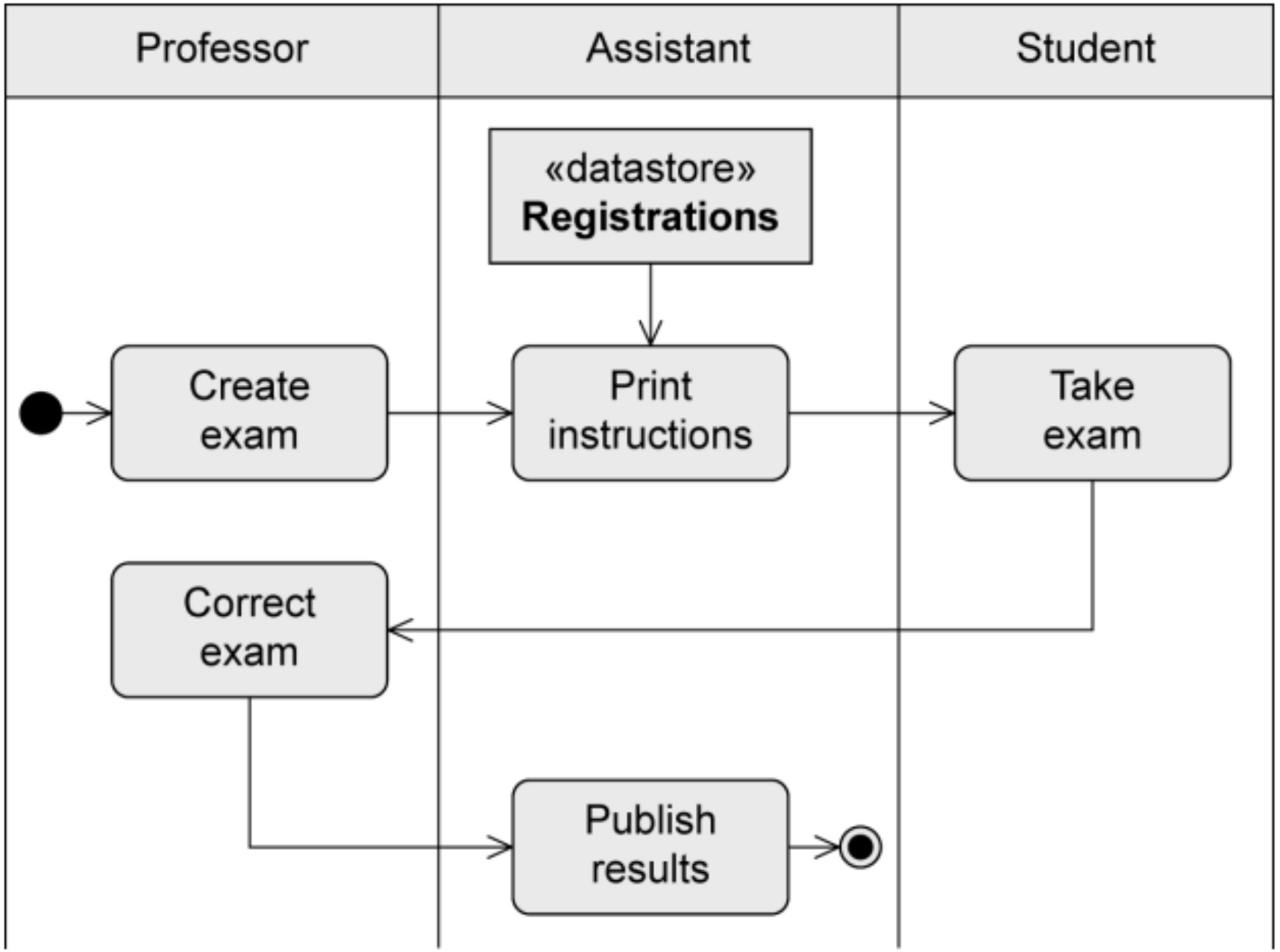
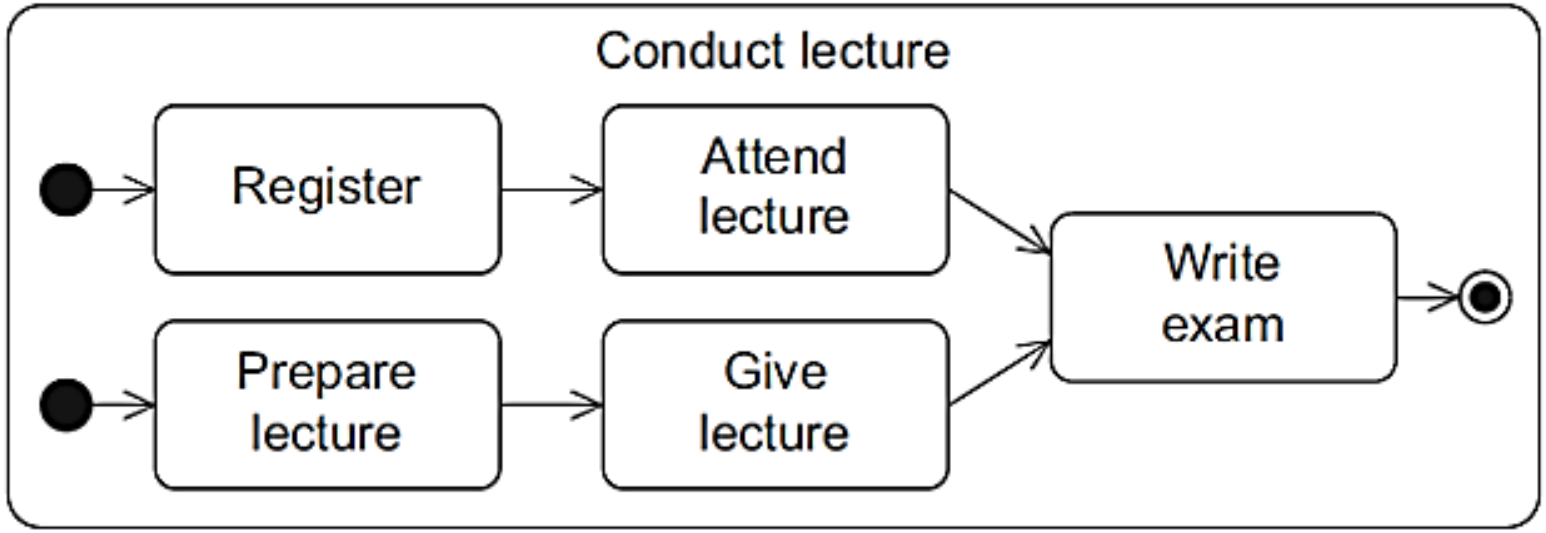


Figure 6

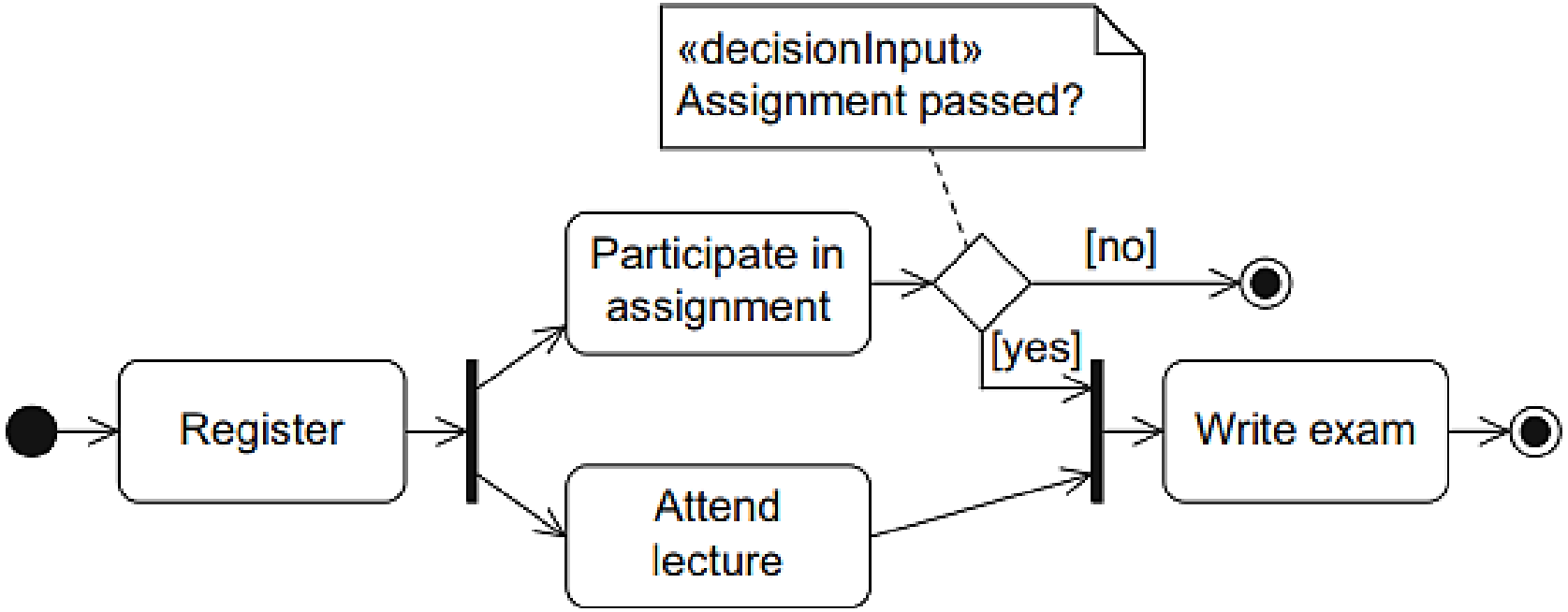
Activity Diagrams: Multiple Initial Nodes

- Multiple initial nodes to model concurrency *(meaning that multiple execution paths can be active simultaneously)*.



- If the activity **Conduct lecture** is activated, the two initial nodes and thus both sub paths are activated. One sub path relates to the actions of students and the other sub path refers to the actions performed by a lecturer. In the action Write exam, both paths are merged.

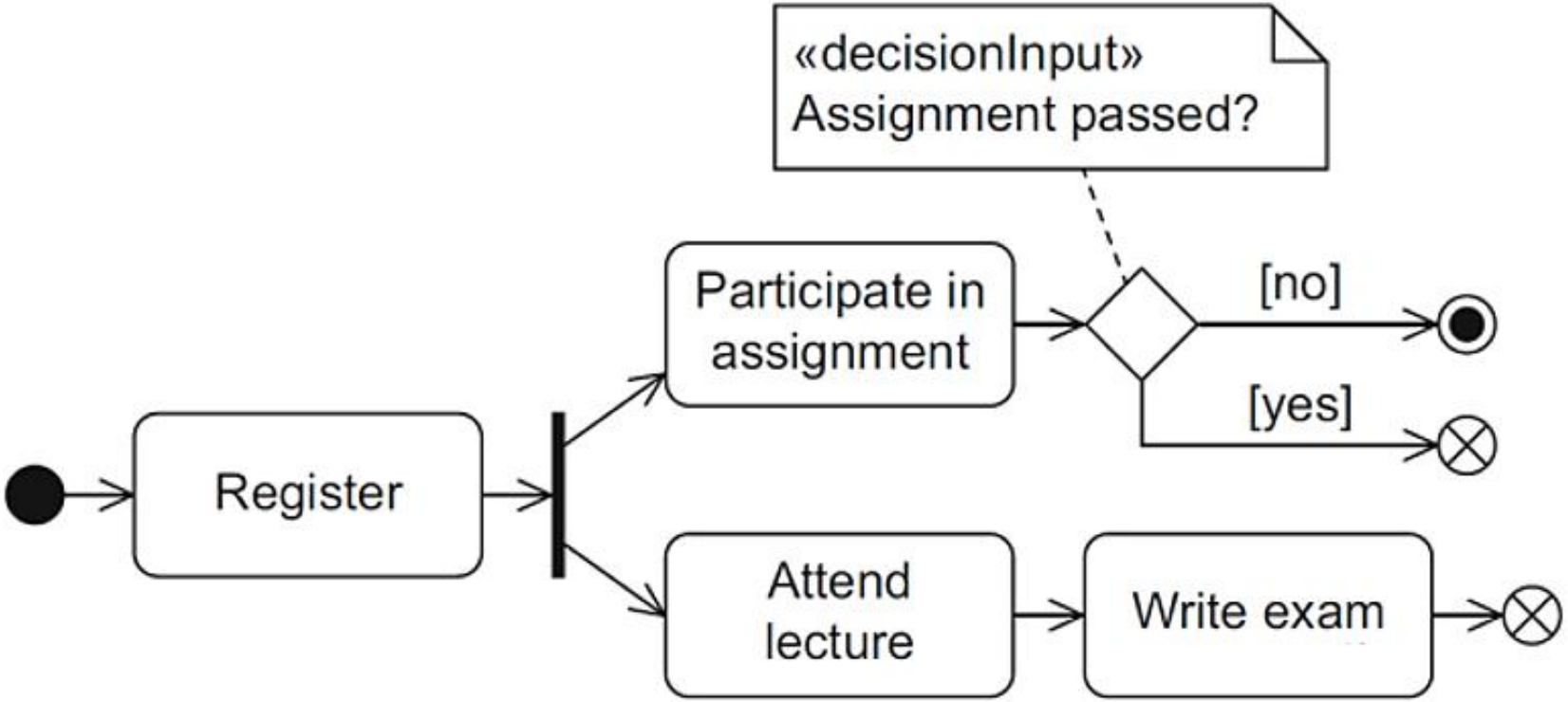
Activity Diagrams: Multiple Final Nodes



- If a diagram contains multiple activity final nodes, the first one reached during the execution ends the entire activity.
- The participation in a course is ended if either the assignment has not been passed or the exam has been taken.

Activity Diagrams: Flow Final Node

- If you only want to end one execution path, leaving the other concurrently active execution paths unaffected, you have to use the flow final node.



An activity diagram of **registering a class**

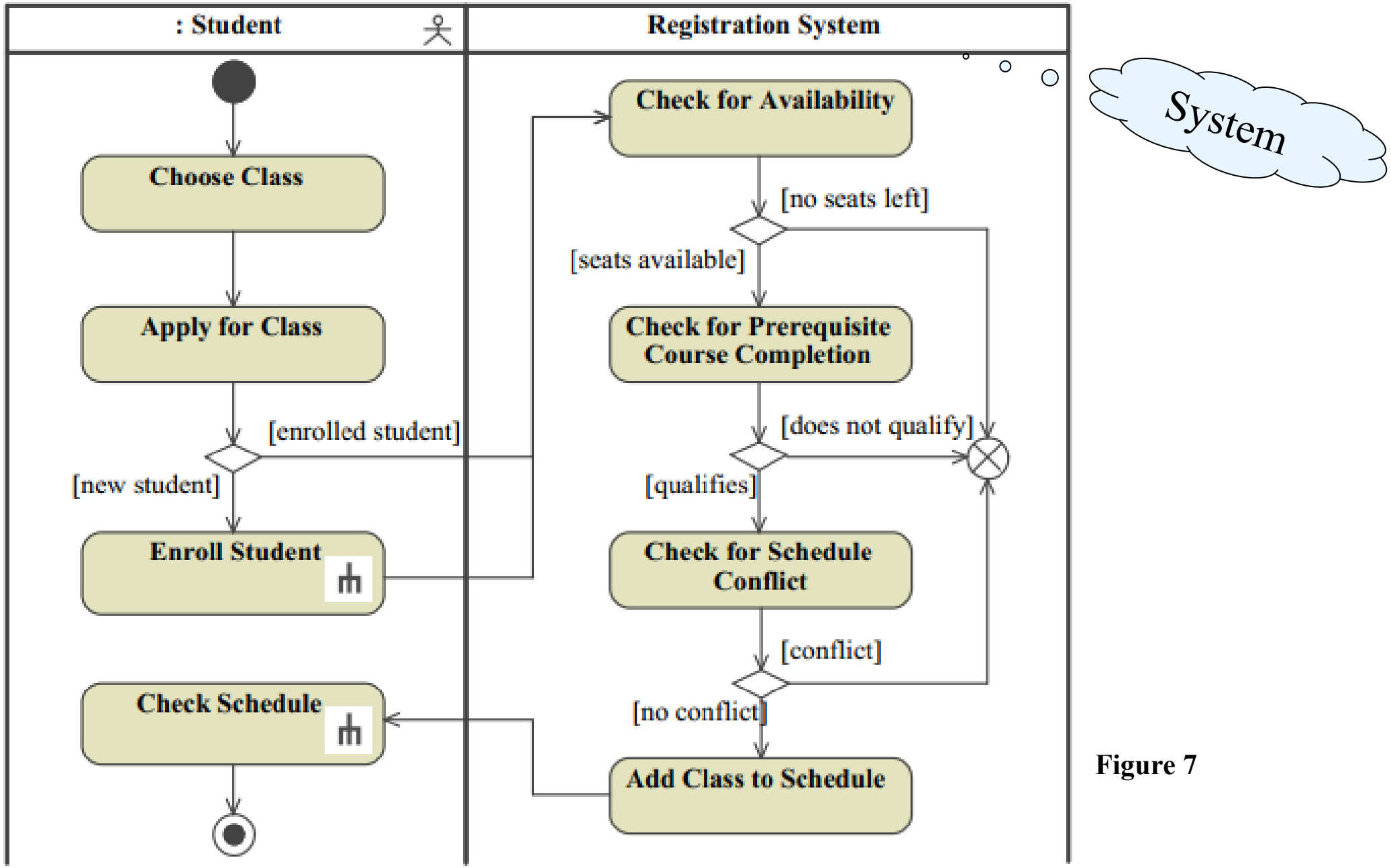


Figure 7

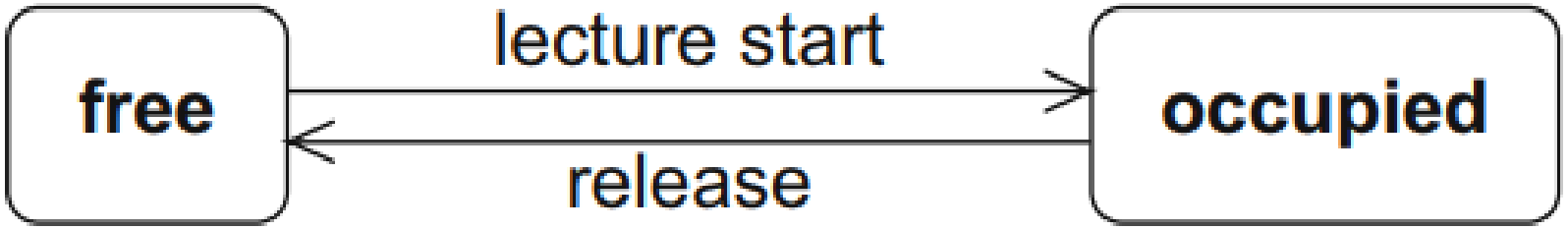
State Diagram

Event-Driven Models

- Event-driven modelling shows how a system responds to **external** and **internal** events.
- A system has a **finite number of states** and that **events** *(stimuli)* may cause a **transition** from one state to another.
- The UML supports event-based modelling using **State Machine Diagram or State Chart Diagram** *(state diagrams)*.
- **State diagrams** show system states and events that cause transitions from one state to another.
- They **do not show the flow of data** within the system.

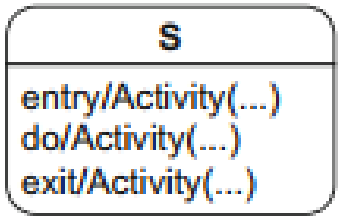
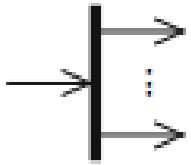
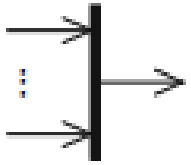
State Diagram: Simple Example

- A state diagram represents how a **single object changes state** over time.



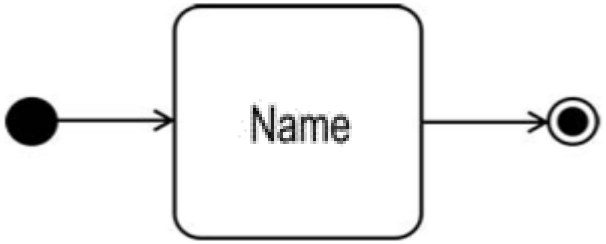
As a simple example consider a ***lecture hall (object)*** that can be in one of two states: free or occupied. When a lecture starts in the lecture hall, the state of the lecture hall changes from free to occupied. Once the respective event in the lecture hall has finished and the hall has been released again.

State Diagram Elements

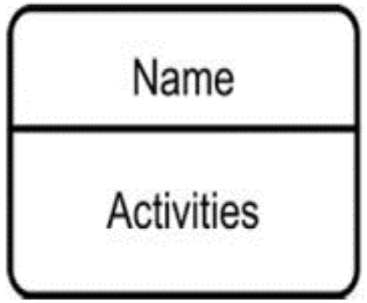
Name	*Notation*	*Description*
State		Description of a specific "time span" in which an object finds itself during its "life cycle". Within a state, activities can be executed on the object.
Transition		State transition e from a source state S to a target state T
Initial state		Start of a state machine diagram
Final state		End of a state machine diagram
Terminate node		Termination of an object's state machine diagram
Decision node		Node from which multiple alternative transitions can proceed
Parallelization node		Splitting of a transition into multiple parallel transitions
Synchronization node		Merging of multiple parallel transitions into one transition

State Diagram

- A state is shown as a rectangle with round corners and is **labeled** with the name of the state.



- If internal activities are specified for a state, it is divided into two sections: the **upper** section contains the **name** of the state; the **lower** section includes **internal activities**.

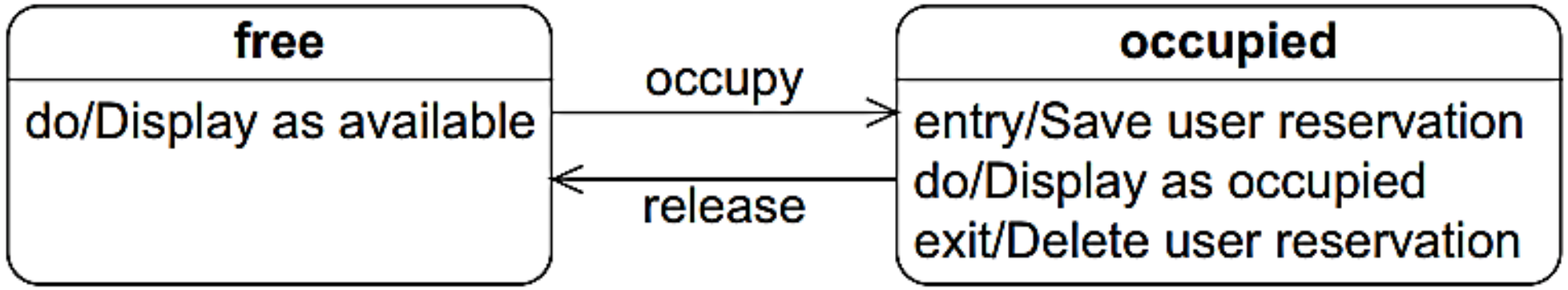


Adding Details to the State Icon

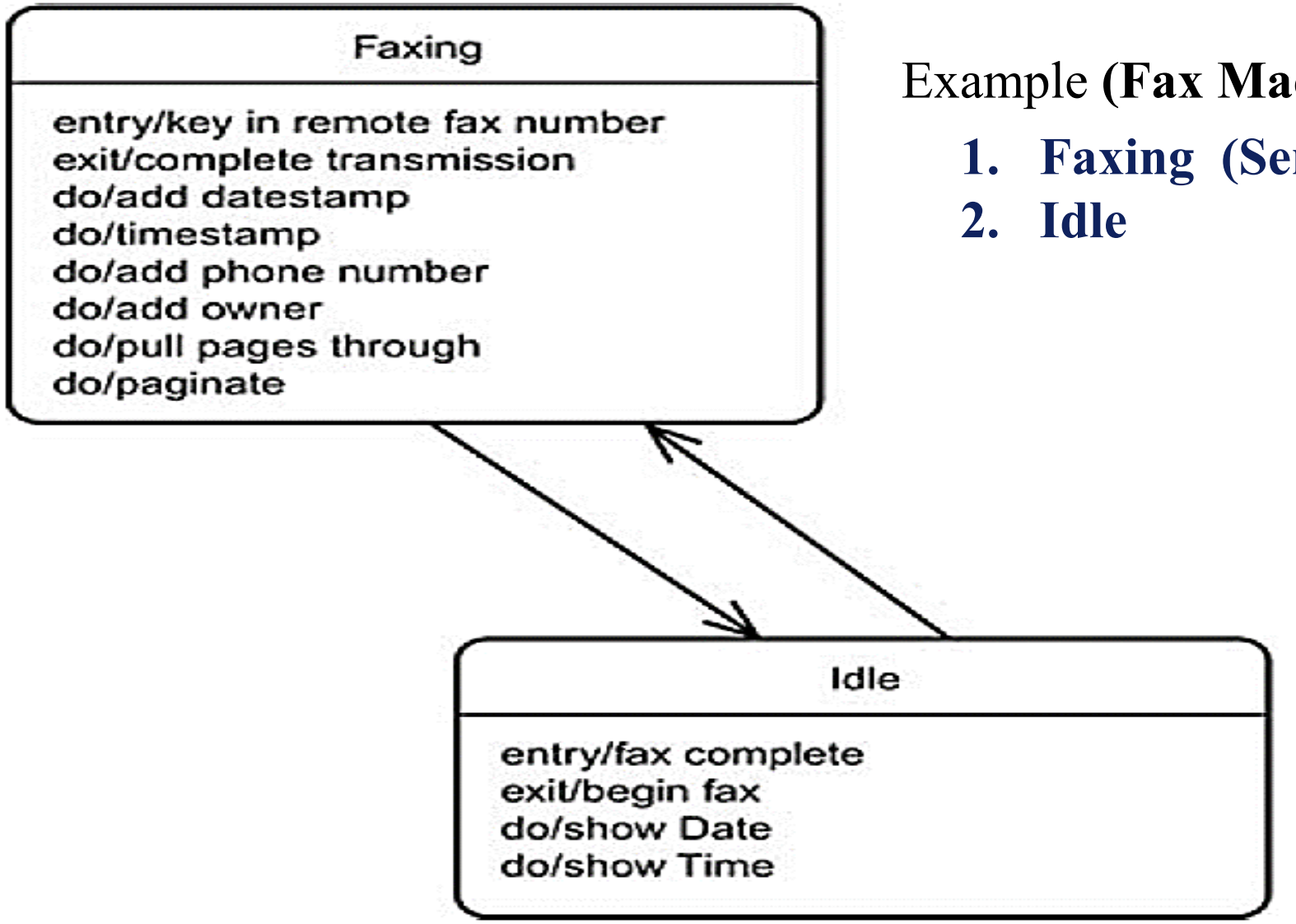
Three frequently used categories of activities :

1. **Entry:** What happens when the system *enters* the state.
2. **Exit:** what happens when the system *leaves* the state.
3. **Do:** what happens while the system is *in* the state.

* **Note**: You can add others as necessary.



Adding Details to the State Icon

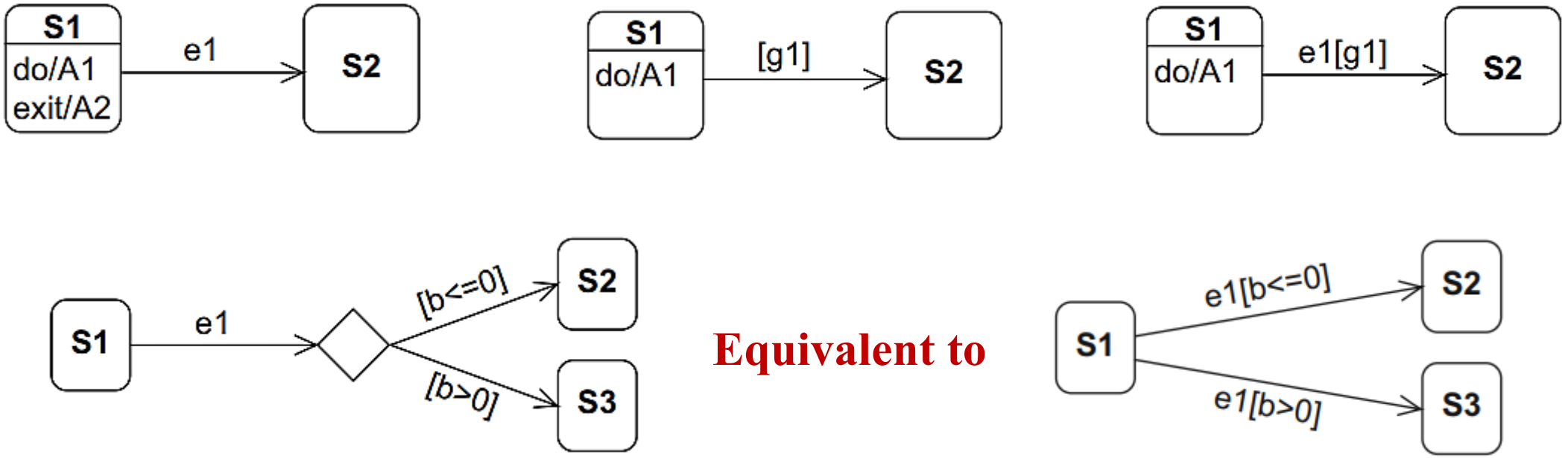


Example **(Fax Machine)** has 2 states:

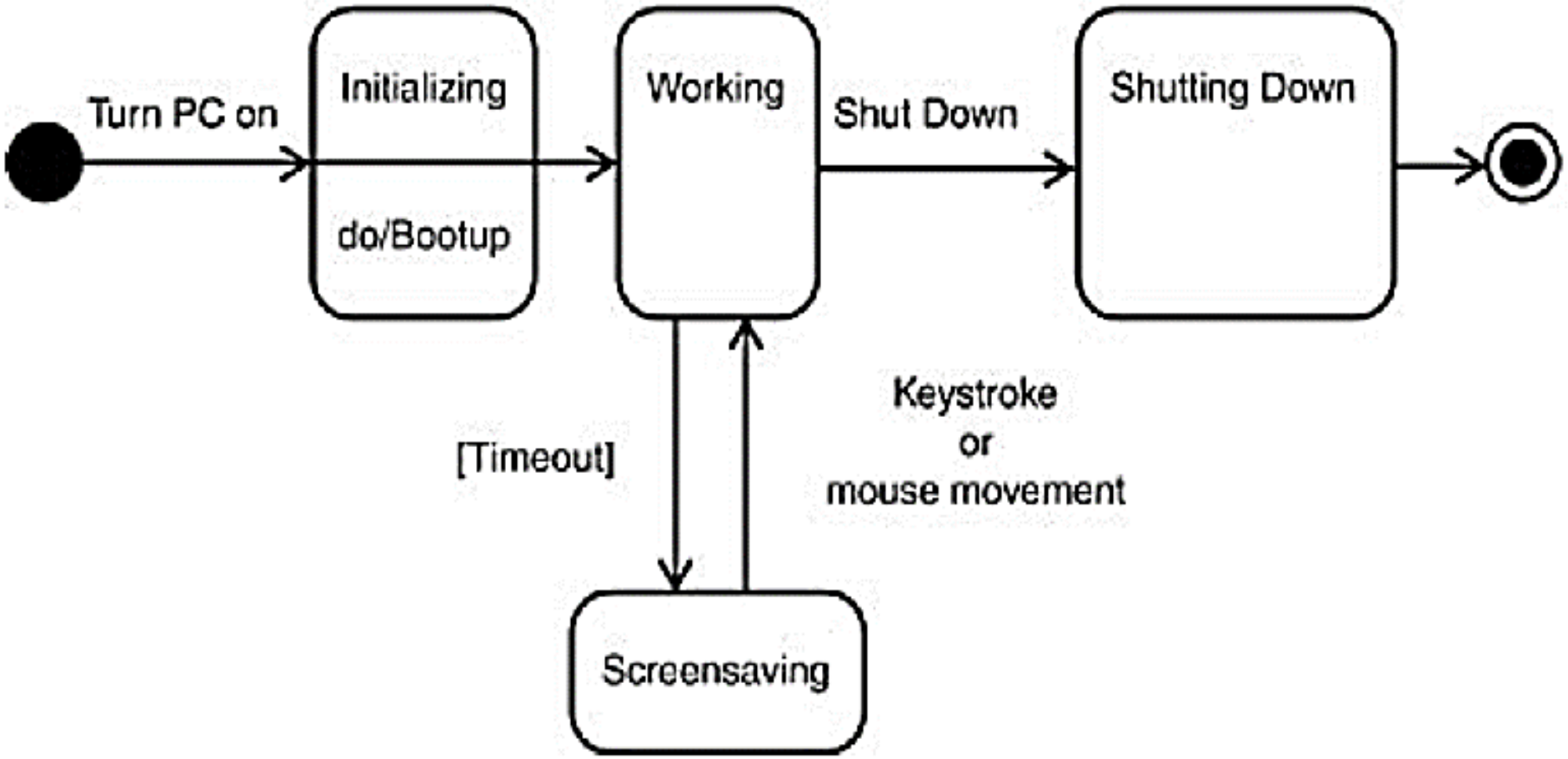
1. **Faxing (Sending a fax)**
2. **Idle**

Adding Details to the Transition

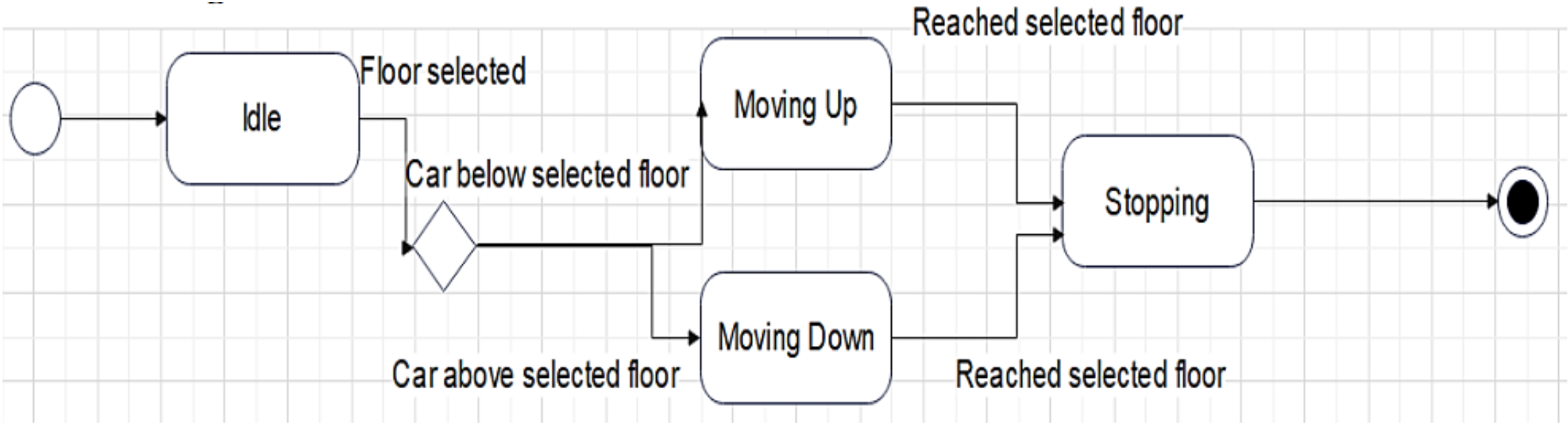
- You can specify various properties for a transition:
  - The **event** (also called "**trigger**") that triggers the state transition.
  - The **guard** (also called "**condition**") that enables the execution of the transition.



Adding Details to the Transition



Adding Details to the Transition



* **hint:** *Car is the Elevator*

Thank You